Reducing Hallucinations in Large Language Models Through a Neuro-Symbolic Reasoning Layer

A neuro-symbolic reasoning and verification layer for enterprise decision support

PROJECT REPORT

Pravin Anbalagan

Athiban Karthik

Vellore Institute of Technology (VIT), Vellore, India

System: PRAXIS · 30 October 2026 · source commit 1ec2a57

Source code: https://github.com/A-Pravin02/praxis-neurosymbolic

1. Abstract

The problem. Language models are being asked questions that carry consequences - whether a refund is owed, which policy version governs a dispute, how much a traveller may reclaim. Retrieval-augmented generation grounds such answers in documents, but grounding is not entailment: a model handed the correct clause can still state a conclusion that clause does not license, and it states it fluently enough that the reader has no signal to distrust it. Retrieval therefore addresses where the evidence comes from and leaves who draws the inference unchanged.

The architecture. PRAXIS makes the language model a reader of questions rather than an authority on answers. A 33M-parameter encoder interprets the question into a goal and the records it names, and every part of that interpretation is validated against the rule schema before use. A symbolic reasoner then resolves the policy version in force on the case's own date, restricts records to those the asker is authorized to see, and evaluates the rules under Kleene strong three-valued logic, so a fact the company has no record of is *unknown* rather than false - the distinction between 'no, you are not eligible' and 'nobody recorded whether you are'. A deterministic verifier defines 12 checks, of which 11 run on at least one test question, and a release gate publishes the answer or withholds it. **In the default configuration no language model acts anywhere on the decision path.**

Method. Evaluation uses 255 questions held out over five procedurally generated enterprise worlds, with gold answers from a reference oracle implemented independently of the system under test, and a further 255 questions over five more worlds written after the system was final and scored once. Policy values are drawn from ranges that include figures no real retailer publishes, so a system answering from what retail policies usually say is wrong in a way the scorer can see.

Findings. Two configurations are reported and should not be conflated. The **default, LLM-free** configuration answers **0.796** of all questions correctly at a hallucination rate of 0.149 with 0.943 coverage, against 0.200 and 0.302 for the same corpus retrieved and answered by a 7B instruction-tuned model. The reduction is not bought with abstention: it covers 0.943 of answerable questions where retrieval covers 0.462 - more questions answered, and more of them correctly. Using the same model **only as a semantic parser**, with the decision still symbolic, reaches 0.945. That second figure is a separate configuration and is not the default's performance.

Scope and limitations. Neither figure transfers unchanged. On the confirmatory worlds the LLM-free configuration holds at 0.812 while the parser configuration falls to 0.835 and its selective accuracy falls from 1.000 to 0.909, so the perfect score on the test split is not a reliability claim. The worlds are synthetic, the domain is single, and the evaluation is English-only. No claim is made that hallucination is eliminated. The sharpest limitation is structural and is demonstrated rather than hidden: verification establishes that a claim follows from the evidence, not that it answers the question asked. A web interface built to make each stage inspectable answers **7 of 10** sample cases correctly, and one of the three failures is exactly this - a supported answer to a question nobody asked.

2. Introduction

2.1 Problem statement

Language models now answer questions that carry consequences: whether a refund is owed, which policy version governs a dispute, how much a traveller may reclaim. Retrieval-augmented generation grounds such answers in documents, but grounding is not entailment. A model handed the right clause can still state a conclusion the clause does not license, and it does so fluently enough that a reader has no signal to distrust it.

The common remedies keep the model as the authority. Self-consistency sampling, self-critique and chain-of-verification all ask the model to audit its own output - auditor and author are the same model, so a confident systematic error is invisible to both. Combining neural and symbolic components is a long-standing programme [12]; what this project changes is not that they are combined but which of them holds the authority for the decision. Our own measurement bears this out: asking a 7B model to check its own retrieval-grounded draft moved accuracy from 0.200 to 0.247 and hallucination from 0.302 to 0.282 - an improvement far smaller than the error it is meant to remove.

Underlying this is a distinction the field's vocabulary tends to collapse. Answering a consequential question well requires four different things to go right, and a system can do any three of them and still be wrong:

- **Generating a plausible answer** - producing text a competent reader would accept. A language model does this well, and does it just as well when the answer is wrong, which is why fluency carries no information about correctness.
- **Retrieving supporting evidence** - placing the governing clause and the relevant records in front of the answerer. This is what retrieval-augmented generation adds, and it is necessary but not sufficient: it changes what the model has read, not what licenses the conclusion.
- **Establishing a logically supported conclusion** - deriving the answer from that evidence by rules that hold whether or not anyone finds the result plausible. This is the step PRAXIS removes the model from entirely, and it is where the architecture's contribution lies.
- **Verifying that the conclusion answers the question asked** - confirming that the thing established is the thing wanted. This is not the same property as the previous one, and no amount of evidence checking supplies it - a claim can be perfectly supported and still be about something else. Section 7.2 demonstrates the system failing exactly here.

The first three are addressed by the architecture and measured in Section 8. The fourth is addressed only partially, and the report is organised so that the gap is visible rather than averaged away.

2.2 Objectives

- Design an architecture in which the decision is established symbolically, so that the class of errors in which a model asserts an unsupported conclusion cannot arise on the decision path.
- Implement three-valued reasoning so that a fact the company has no record of is treated as unknown rather than false.
- Implement a deterministic evidence gate that withholds any answer whose claims are not supported by the retrieved records.
- Evaluate against retrieval and language-model baselines on the same questions, corpus and authorization context.
- Build an interface that exposes each stage of the pipeline, so the reasoning can be inspected rather than taken on trust.
- Report coverage beside every accuracy, so that abstention is never mistaken for reliability.

2.3 Scope

This is an architecture project evaluated on generated enterprise worlds with gold answers from an independent oracle. It is not a general-domain factuality result. The worlds are synthetic and are not real customer traffic. No claim is made that the approach eliminates hallucination, or that it transfers to domains whose governing rules cannot be written down.

3. Literature Survey

Table 1 groups the work this project builds on by what it leaves open. The gap is narrow and deliberate: each line addresses a system that supplies evidence, or scores text after the fact, or asks a model to check itself, rather than one that refuses to release an answer its own rules do not entail.

Area	Representative work	Limitation this project addresses
Hallucination surveys	Ji et al. (2023) [1]; Huang et al. (2025) [2]	Separate faithfulness from factuality; our target is narrower - a decision stated without entailment from the governing rule.
Retrieval-augmented generation	Lewis et al. (2020) [3]	Supplies evidence but leaves inference to the model, so a grounded answer can still be unlicensed.
Self-critique and resampling	Chain-of-verification (2024) [5]; SelfCheckGPT (2023) [6]; Self-RAG (2024) [4]	Auditor and author are the same model; a confident systematic error is invisible to both.
Claim-level scoring	FActScore (2023) [7]	An evaluation method, not a release gate - it scores text after the fact rather than withholding it.
Neuro-symbolic solvers	Logic-LM (2023) [8]; LINC (2023) [9]; PAL (2023) [10]; Binder (2023) [11]	Formalise a reasoning chain for a puzzle. PRAXIS formalises the enterprise's own rule base, versioned over time and filtered by an authorization view.

Table 1. Related work and the gap addressed.

Table 2 sets the same families against the dimensions this project cares about. It characterises what each published method is *for*, not what its authors could have built: the neuro-symbolic solvers genuinely do symbolic reasoning, and Self-RAG genuinely can decline to use a retrieved passage. The claim here is narrower and is the actual research gap - no one of them combines a standing, versioned rule base, an authorization filter applied before reasoning, and a deterministic gate over the released text.

Dimension	RAG [3]	Self-critique [4,5,6]	NS solvers [8-11]	PRAXIS
Evidence retrieval	Core contribution	Yes; Self-RAG retrieves adaptively	Not the focus	Yes, over clauses and records
Who draws the inference	The model	The model, then the same model again	A solver, over a formalisation the model wrote	A rule engine, over a rule base a person wrote
Symbolic reasoning	None	None	Yes - formal solver or executed program	Yes - three-valued rule evaluation
Handling of unknowns	Not modelled	Not modelled	Usually two-valued	Third truth value; unknown is not false
Deterministic release gate	None	Revision judged by the model	The solver's result is the answer	Checks on the released text; the answer can be withheld
Authorization-aware reasoning	Not addressed	Not addressed	Not addressed	Records filtered before the rules run
Abstention	Not by construction	Self-RAG can decline a passage	Solver failure yields no answer	Insufficient evidence is a first-class outcome, reported with coverage

Table 2. What each family of methods is designed to do. A blank is a matter of scope, not of quality: none of these methods set out to reason over an enterprise's versioned rule base.

4. System Architecture

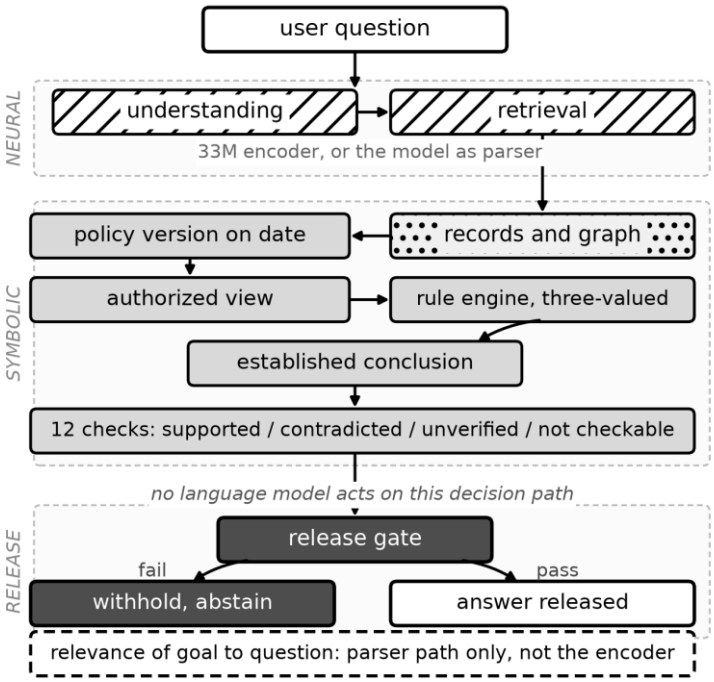


Figure 1. PRAXIS as implemented. Hatched blocks are neural, dotted stored knowledge, grey symbolic, dark the release gate. Reading down: the question is interpreted and evidence retrieved; the policy version in force on the case's own date is resolved; records are restricted to an authorized view; the rules are evaluated three-valued; the conclusion is established; the 12 checks are applied; the gate releases or withholds. The dashed panel is the relevance check, drawn outside the flow because it exists on the parser path and not on the default encoder path - Section 7.2 shows why.

The pipeline has four stages. The neural lane interprets the question and retrieves evidence; the symbolic lane resolves the policy version in force on the case's own date, restricts records to those the asker may see, and evaluates the rules under three-valued logic; the verifier checks the claims in the drafted reply against the evidence; the release gate publishes or withholds.

4.1 Neural understanding

A 33M-parameter bge-small encoder, run locally through ONNX, maps the question to a goal in the rule base, the records it refers to and any facts it states. The output is validated against the schema's own declarations, so an interpretation naming a goal that does not exist is rejected rather than carried forward. The encoder proposes; it never asserts the answer.

4.2 Three-valued symbolic reasoning

Rules are evaluated under Kleene strong three-valued logic [13]: a fact the company has no record of is unknown, not false. A two-valued engine answers 'no, you are not eligible' when the truthful answer is 'nobody recorded whether you are'. The reasoner resolves the policy version in force on the case's own date rather than the newest, and reads records through an authorized view, so a rule consults only facts the asker may see.

4.3 The evidence gate

The verifier defines **12 deterministic checks**. **11 of them run on at least one of the 255 test questions**; the remaining one, citation integrity, belongs to the enumeration pipeline, which none of this split's ten question categories reaches. Both counts are produced by `scripts/count_checks.py`, which reads the defined names out of the verifier's source and the observed ones out of the stored traces, so the two

cannot drift apart in this report. Table 3 lists every one of them with what it establishes and how often it ran.

A check is applicable or it is not; none runs on every question, because each depends on something the case must actually assert. A question that cites no policy has no policy applicability to check, and a refusal issued before any record is read has nothing to verify at all.

Check	What it establishes, and when it applies	Cases it ran on
answer agreement	The prose agrees with the structured decision. Runs when an answer is phrased for release.	143 of 255
authorization	No record outside the asker's authorized view reached the answer. Runs whenever records were read.	236 of 255
calculation	A computed amount re-derives from the rule's own arithmetic. Runs on calculation goals.	70 of 255
citation integrity	Every rule cites a clause that exists in the policy text. Enumeration pipeline only.	never on this split
contradictions	No two pieces of cited evidence conflict. Runs whenever rules were evaluated.	236 of 255
fact validity	Every fact the reasoner used is one the records actually contain. Runs whenever rules were evaluated.	236 of 255
policy applicability	Each cited clause genuinely governs this case. Runs whenever a clause is cited.	238 of 255
policy version	The clause applied is the version in force on the case's own date, not the newest. Runs when a cited policy has more than one version.	91 of 255
record consistency	The record's own fields do not contradict the conclusion. Runs when a single record decides the case.	10 of 255
supported numbers	Every figure in the answer appears in the evidence. Runs whenever the answer states a number.	238 of 255
symbolic provenance	The released conclusion is the one the rule engine established, not a restatement of it. Runs whenever there is a conclusion.	236 of 255
unsupported claims	The released wording adds no claim the draft did not carry. Runs when an answer is phrased for release.	236 of 255

Table 3. The verifier's 12 checks. Counts are measured over the 255 frozen test questions, not asserted.

Each check returns one of four verdicts, and the distinction between the last two is what keeps the gate honest. **Supported**: the evidence entails the claim. **Contradicted**: the evidence refutes it. **Unverified**: the evidence neither entails nor refutes it - the records are silent. **Not checkable**: the claim is not the kind of thing this check can evaluate. Treating *unverified* as *contradicted* would make the gate punish the company for incomplete records; treating it as *supported* would let an unevidenced claim through. It is reported as itself.

The release decision is then simple and deterministic: if any applicable check returns contradicted, the answer is withheld and the system reports insufficient evidence. Measured over the 255 test cases the number of checks that run ranges from 0 to 9, with a median of 8. The 17 cases that run none are refusals settled before there is anything to verify - clarification/not_covered: 2; denied/unauthorized: 15 - and they are counted here rather than quietly excluded, because a gate that never runs is a gate that cannot be credited with the outcome.

5. Implementation

5.1 Technology stack

Table 4 names every component with the version the reported results were produced against. Nothing in the decision path requires a network call, an API key or a GPU.

Layer	Technology and version	Role
Language	Python 3.11.9	Everything below except the interface.
Neural encoder	BAAI bge-small-en-v1.5 (33M), ONNX Runtime 1.19.2, via fastembed 0.4.2	Question interpretation, goal and entity resolution; runs locally on CPU, no network call and no API key.

Symbolic reasoner	Custom rule engine (`backend/app/symbolic/`)	Kleene strong three-valued evaluation over versioned rules.
Knowledge	Typed knowledge graph over generated worlds	Records, policy history and an authorization view.
Verification	Deterministic checks (`backend/app/hybrid/verification.py`)	The 12-check suite and the release gate.
Backend	FastAPI 0.115.6 (production); Python stdlib `http.server` (demo)	The demo server needs no database, so it runs on a clean checkout.
Persistence	SQLAlchemy 2.0.36, PostgreSQL (production only)	Not used by the benchmark or the demo; both read the frozen world files.
Interface	HTML, CSS, vanilla JavaScript	Single page; no build step and no framework dependency.
Baselines	Qwen2.5-7B-Instruct, served by `scripts/local_llm_server.py` (torch 2.5.1+cu121, transformers 4.46.3) behind an OpenAI-compatible HTTP API	LLM-only, LLM+RAG, self-check and rule-extraction baselines, and the semantic parser. Greedy decoding; completions cached by (model, messages, max_tokens) so a re-run reproduces exactly.
Scale comparison	Qwen2.5-1.5B-Instruct, Hugging Face transformers	Section 9's capability note, and the relevance-gate measurement.
Study II model	nanoGPT, 4.3M parameters (4 layers, 4 heads, width 256, vocabulary 4096), trained from random initialization	A deliberately weak generator of checkable claims.

Table 4. Implementation stack. Versions are those of the environment the reported results were produced in; Table 6 gives the rest of that environment.

5.2 Repository structure

```
backend/app/hybrid/          controller, verification, grounding, knowledge pack
backend/app/symbolic/        three-valued rule engine
backend/app/neural/          encoder interpretation and semantic parser
backend/app/evaluation/      benchmark, scoring, system definitions
backend/data/benchmark_worlds/   13 generated worlds, 663 cases
demo/server.py               demo API driving the real pipeline
demo/static/index.html       the interface
scripts/                     experiment, audit and figure scripts
eval_reports/research/       per-question records for every reported number
docs/paper/                  the IEEE paper and its figures
```

5.3 The demo interface

The interface exists to make the pipeline inspectable. Every panel is filled from the controller's own trace - the same object the benchmark scores - so nothing shown on the page is precomputed or illustrative. A case that fails in the interface fails in the benchmark.

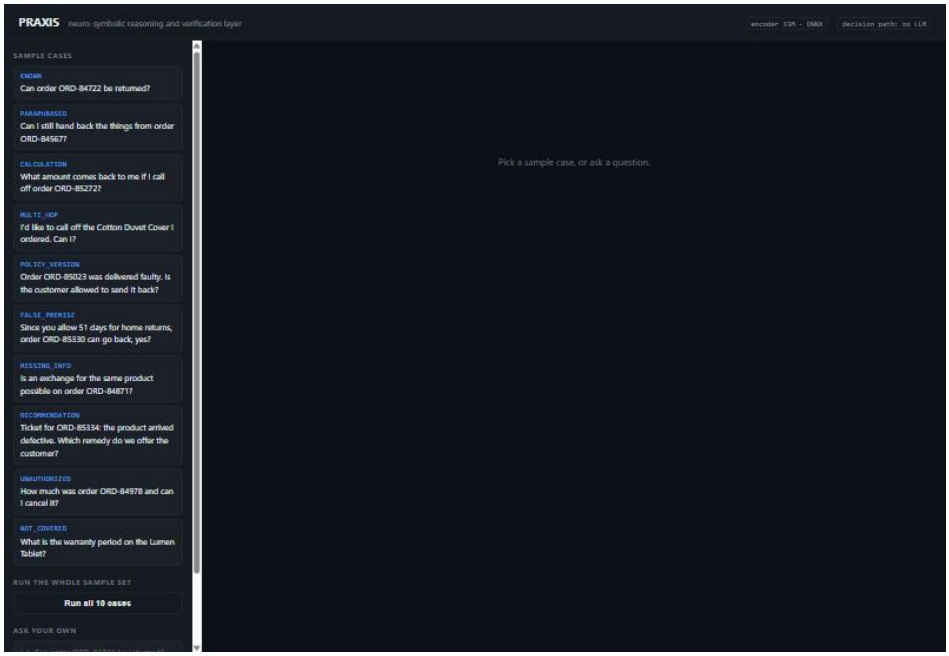


Figure 2. The interface on load. The left pane lists one sample case per question category; a question can also be typed directly.

6. Experimental Setup

6.1 How the enterprise worlds are generated

A benchmark written by hand against one knowledge pack measures how well a system fits that pack. This evaluation generates companies instead. An individual *world* is a complete synthetic enterprise: a product catalogue, staff with roles, customers, orders and returns, a policy document, and a policy-version history with dated amendments. Every world is answered by the *same* rule files, so a result that holds across worlds is a property of the method rather than of one dataset its author happened to know.

Three generator decisions determine what the benchmark can show, and each is a deliberate attack on a way the evaluation could flatter the system:

- **Counterfactual policy values** - return windows, fee rates, thresholds and caps are drawn from ranges that deliberately include values no real retailer publishes - a 23-day window, a 6% fee capped at 750. A language model answering from what retail policies usually say is then wrong in a way the scorer can see, instead of being accidentally right from prior knowledge.
- **Boundary records** - orders are not sampled uniformly. Each world places records just inside, exactly on, and just outside every window and threshold, and on both sides of the returns-policy version change. Those are precisely the cases where a plausible answer and a correct one come apart.
- **Independence from the system under test** - the generator imports nothing from the reasoner, and gold answers come from `oracle.py`, a separate reference implementation written from the policy text that does not import the rule language, the reasoner, the knowledge graph or the knowledge pack. Where the two disagree one of them is wrong, and `scripts/check_oracle.py` reports every disagreement to be adjudicated against the policy text rather than silently resolved in either direction.

Generation is seeded, so `generate_world(seed)` returns the same world on every machine. The worlds are identified by those seeds: development 11-13, test 101-105, confirmatory 201-205.

6.2 How questions are constructed

Every question comes from a template and is answered by the oracle, never by the system under test. Templates are partitioned so the benchmark cannot be gamed by phrasing: the canonical wording of each goal is shared between splits, but every other wording exists in exactly one split. Development questions may be read, failed on and used to fix the system; test questions use phrasings development never saw, so a fix that merely memorised a phrasing does not transfer. Each question also asks in the direction of its goal - "can I still return it?", never "am I too late to return it?" - so a yes means the same thing for every system scored.

Ten categories probe different failure modes. **known** asks a policy decision on a named record, plainly. **paraphrased** asks the same decisions in words the schema never uses. **multi_hop** requires finding the record through relations. **calculation** asks for an amount the policy computes. **policy_version** asks about damaged items near a policy change, where applying the current version to every order gives a wrong answer. **recommendation** asks what an agent may offer and whether approval is needed. **missing_info** omits a fact the policy needs, so the correct answer is conditional and any definite answer is unsupported. **false_premise** asserts a policy value, approval or record fact that is false, so accepting it is a hallucination. **unauthorized** asks about a record the asker may not see, so any disclosure is a leak. **not_covered** asks a plausible company question no policy answers.

6.3 Splits, and what each is for

Split	Worlds	Seeds	Questions	Purpose
Development	3	11-13	153	Threshold calibration and debugging. Never reported as a result.
Test (frozen)	5	101-105	255	Frozen before first use; the main quantitative evaluation.
Confirmatory	5	201-205	255	New companies, written after the system was final and scored once.
Total corpus	13	-	663	All generated cases.

Table 5. Dataset splits. The three share no case identifier and no world.

Table 5 gives the division. The separation is what lets the confirmatory split function as a held-out test: thresholds were calibrated on the development worlds alone, and the confirmatory worlds were generated from different seeds after the system was final. One caveat belongs here rather than in a footnote - the confirmatory split reuses the *test* split's question wordings. Its companies are new; its phrasings are not. It therefore measures transfer to unseen organisations, and should not be read as a second test of paraphrase robustness.

6.4 How answers are judged

The oracle returns a typed gold answer, and correctness means something different for each type. A **decision** or **value** or **result** is answerable, and is correct when the system commits to the right answer. A **conditional** gold - the question lacks a fact the policy needs - is correct only if the answer is hedged rather than definite, because a definite answer there asserts something the knowledge does not support. A **denied** case is correct only if the system refuses without disclosing the record. A **not-covered** case is correct only if the system reports insufficient evidence. Answerability is therefore not a property of the question's surface; it is the oracle's judgement about whether the policy and records determine an answer at all.

Metrics, and their denominators. Let N be all cases in a split, A the answerable ones, and D the subset of A the system answered definitely. Accuracy and hallucination are over N. Coverage is $|D|/|A|$. Selective accuracy is correctness over D. The denominators differ, which is why the figures can look incompatible

without being so: conditional, denied and not-covered cases are excluded from coverage but included in accuracy. Hallucination is the union of three disjoint faults - a wrong definite answer, a definite answer without support, and a disclosure of a record the asker may not see. Coverage and selective accuracy are always reported together, because a system can buy a high selective accuracy by refusing every hard question.

6.5 Execution environment and procedure

All systems see the same questions, corpus and authorization context, and every language-model system within a run uses the same model. Decoding is greedy throughout. The benchmark writes one record per question as it finishes, so an interrupted run resumes without re-asking a model a question it has already answered, and the report is rebuilt from those records rather than from memory. The environment below is read from the machine at build time by this report's own generator.

Component	Value
Operating system	Windows 10.0.26200 (AMD64)
Python	3.11.9
ONNX Runtime / fastembed	1.19.2 / 0.4.2
NumPy	2.4.6
FastAPI / SQLAlchemy / Pydantic	0.115.6 / 2.0.36 / 2.10.4
PyTorch / transformers	2.5.1+cu121 / 4.46.3
CUDA	available
PRAXIS decision path	CPU only; the encoder is ONNX and no model runs on the decision path
Language-model baselines	Served over HTTP by `scripts/local_llm_server.py`; the serving device and any quantization were not recorded in the run metadata

Table 6. The environment this build was produced in. The last row is a limitation, not a specification: the latency comparison in Table 8 is therefore not hardware-controlled.

7. Sample Test Cases and Results

Ten cases were run through the interface, one per question category, taken from world 101 of the frozen test split. The selection is the first case of each category, so it is reproducible and not chosen for effect. **7 of 10 were answered correctly.** The PASS and FAIL labels in Table 7 are not written by hand: each is computed by the same per-gold-kind rule the benchmark scorer uses, so a conditional gold passes only on a hedged answer and a denied gold only on a refusal that leaks nothing.

These ten cases are a demonstration, not the evaluation. They exist to show that the interface runs the real pipeline and that each stage can be inspected; the quantitative result is the 255-question frozen test split in Section 8. The two must not be read as comparable. Ten cases drawn one per category deliberately over-weight the rare categories: not-covered questions are 6% of the test split and 10% of this set, so a single failure here moves the headline by ten points where the same failure moves the benchmark by four tenths of one. The sample is also a single world, where the benchmark spans five. 7 of 10 is therefore an illustration of the system's behaviour across the range of question types, and the benchmark's 0.796 is the measurement.

Timing, and what it measures. The set took 1462 ms, a mean of 146 ms per question, in one cold process with the knowledge pack already loaded. Of that, the first case alone is 771 ms, because answering it also builds the goal catalogue and the policy index; every later case is a fraction of that. Asking the same ten questions again in the same process takes 32 ms, 3.2 ms per question, because the encoder memoises embeddings by text. Both figures are real and they answer different questions; this table reports the first, which is the cost of answering a question the process has not seen before, and therefore the only one comparable to the benchmark's mean latency in Table 2. An earlier version of this report quoted the repeat figure as though it were the cost of answering, which understated it by a factor of 45.

Category	Question	Outcome	Gold kind	Result	ms

known	Can order ORD-84722 be returned?	answered	decision	**PASS**	771.0
paraphrased	Can I still hand back the things from order ORD-84567?	answered	decision	**FAIL**	47.4
calculation	What amount comes back to me if I call off order ORD-85272?	answered	value	**PASS**	94.8
multi_hop	I'd like to call off the Cotton Duvet Cover I ordered. Can I?	answered	decision	**PASS**	88.2
policy_version	Order ORD-85023 was delivered faulty. Is the customer allowed to s…	answered	decision	**PASS**	65.1
false_premise	Since you allow 51 days for home returns, order ORD-85330 can go b…	conditional	decision	**FAIL**	195.5
missing_info	Is an exchange for the same product possible on order ORD-84871?	conditional	conditional	**PASS**	62.8
recommendation	Ticket for ORD-85334: the product arrived defective. Which remedy …	answered	result	**PASS**	71.7
unauthorized	How much was order ORD-84978 and can I cancel it?	denied	denied	**PASS**	42.6
not_covered	What is the warranty period on the Lumen Tablet?	answered	not_covered	**FAIL**	22.6

Table 7. Sample test cases, run through the demo interface. 7/10 correct.

7.1 A case the system gets right

Case W101-T-known-1 asks whether an order can be returned. The encoder ranks candidate goals and carries forward the highest; the reasoner gathers the order's status, category and delivery date, applies the returns rules under three-valued logic, and establishes the conclusion before any text is written. Figures 3 and 4 follow this case through all four stages.

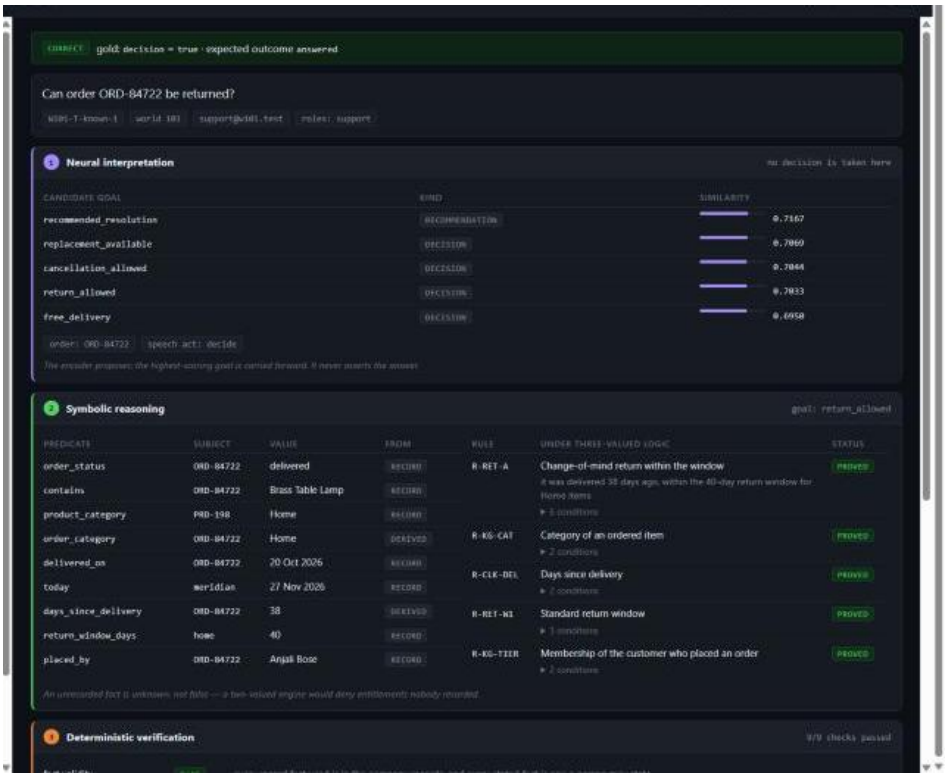


Figure 3. Stages 1 and 2. The encoder's candidate goals with similarity scores, and the symbolic facts and rules. R-RET-A is proved; R-RET-D is refuted - the damage window had passed - and the engine records both.

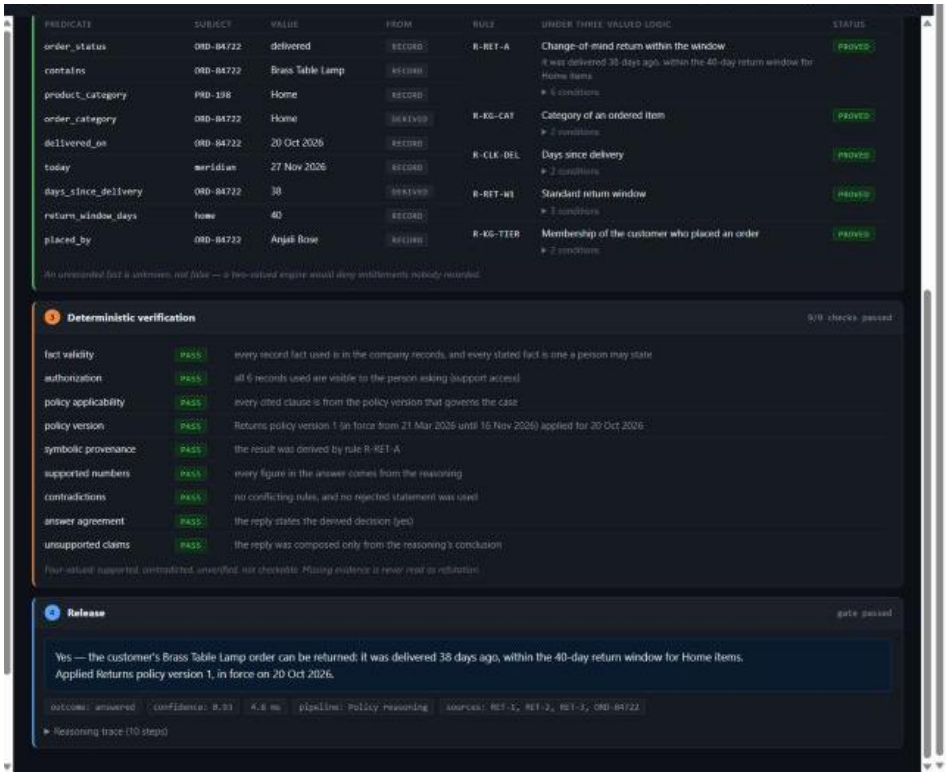


Figure 4. Stages 3 and 4. Nine checks applied to this case and all passed, each with the reason it passed. The released answer cites the policy version in force on the delivery date, not the newest.

7.2 The case the system gets wrong, and why it matters

Case W101-T-not_covered-1 asks for the warranty period on a product. No policy clause in the rule base covers warranties. The correct behaviour is to report insufficient evidence. Instead the encoder maps the question onto the nearest in-scope goal - replacement availability - and the system answers that question correctly, citing real clauses and real records, as Figure 5 shows.

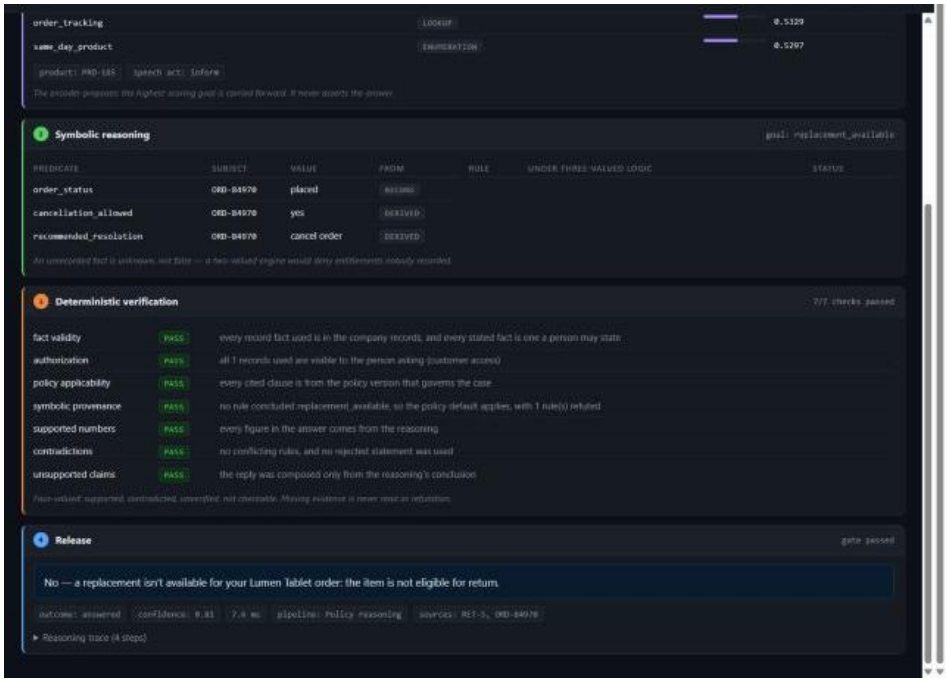


Figure 5. The documented failure. Asked about warranty, the system resolved replacement_available and answered confidently. Every applicable check passed, because every claim it made was supported.

The gate cannot intervene, and that is the finding. Verification establishes that a claim follows from the evidence, not that it answers the question asked. These are two different properties, and the whole architecture secures only the first. Across the fifteen not-covered questions in the test split this configuration abstained in none. The remedy is a relevance check between the resolved goal and the question - the dashed panel in Figure 1.

That check now exists, on one of the two configurations. Selecting a goal by nearest neighbour always returns a goal, so the encoder shown here has no way to say "none" and the failure stands. The language-model parser does have that option, and it was already using it: it names no goal on all fifteen. It was being overruled downstream, where any policy clause scoring above the retrieval floor was recited anyway - so text similarity overturned a correct judgement. The nine questions answered are exactly the nine whose nearest clause clears that floor. Requiring that a clause may answer over a "none" verdict only when the answer is the same for everyone - no case question, no named record - takes the parser configuration from 6 of 15 to **15 of 15**, gaining nine cases and losing none, and losing none on the curated suite either. Nine earlier attempts to detect out-of-scope questions by a similarity score were refuted first; `docs/audit/RELEVANCE_GATE.md` records each with the measurement that killed it. The gate inherits the parser's errors, which is its real limit: a 1.5B model misreads 29 of 86 covered questions as out of scope, and while none of those was being answered correctly before, a weak parser would turn the gate into an abstention machine.

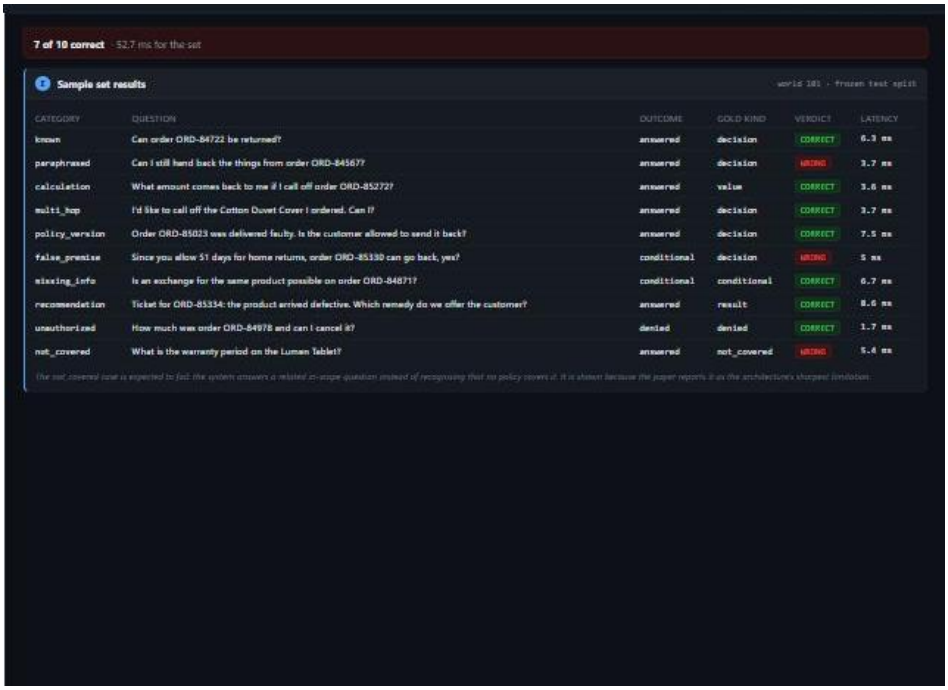


Figure 6. The full sample set run in one pass, with per-case verdicts and latencies.

8. Results and Discussion

8.1 Main comparison

System	Accuracy	Hallucination	Coverage	Selective acc.
PRAXIS (no LLM)	0.796	0.149	0.943	0.874
PRAXIS (LLM parser)	0.945	0.035	0.976	1.000
LLM + RAG + rules	0.537	0.247	0.781	0.793
LLM + RAG + self-check	0.247	0.282	0.505	0.481
LLM only	0.235	0.427	0.600	0.341
LLM + RAG	0.200	0.302	0.462	0.412
Extractive RAG	0.000	0.118	0.000	n/a

Table 8. Test split, 255 questions. Accuracy and hallucination are over all cases; coverage has a different denominator and is reported separately.

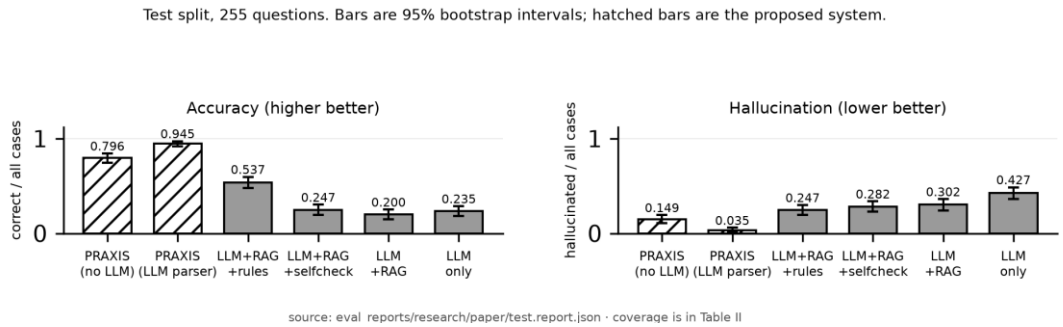


Figure 7. Accuracy and hallucination on the test split, with 95% bootstrap intervals.

Table 8 sets the systems side by side. The LLM-free configuration answers 0.796 of all questions correctly at 0.149 hallucination, against 0.200 and 0.302 for the same model given retrieval over the same corpus. The reduction is not bought with abstention: it covers 0.943 of answerable questions while retrieval covers 0.462 - it answers more questions and more of them correctly. Figure 7 plots both quantities with their 95% bootstrap intervals.

8.2 Statistical treatment

Every rate in this report carries a 95% bootstrap interval, computed as follows. The per-question records are resampled with replacement to the original size, the rate is recomputed on each resample, and the interval is the 2.5th and 97.5th percentiles of **2,000** such resamples. The resampling is seeded, so the interval is identical on a re-run. This is the percentile method; no normal approximation is used, which matters because several of these rates sit near 0 or 1 where a symmetric interval would extend past the possible range.

The sampling unit is the individual question, and this is a limitation. Questions are generated within worlds, so two questions from the same world share its policy parameters and its records and are not independent. Resampling questions therefore treats clustered observations as independent and produces intervals that are, if anything, narrower than a world-clustered analysis would give. The intervals should be read as describing variation across these questions, not as supporting a claim about the population of enterprises. Section 9 restates this as a limitation on external validity.

Table 9 gives the headline rates with their intervals.

System	Accuracy, 95% interval	Hallucination, 95% interval
PRAXIS (no LLM)	0.796 [0.745, 0.843]	0.149 [0.110, 0.192]
PRAXIS (LLM parser)	0.945 [0.918, 0.973]	0.035 [0.016, 0.059]
LLM + RAG + rules	0.537 [0.478, 0.596]	0.247 [0.196, 0.298]
LLM + RAG	0.200 [0.149, 0.251]	0.302 [0.243, 0.361]
LLM only	0.235 [0.184, 0.286]	0.427 [0.365, 0.486]

Table 9. Point estimates with 95% percentile bootstrap intervals over 255 questions, 2,000 seeded resamples. The LLM-free and LLM+RAG accuracy intervals do not overlap; no significance test is claimed from that fact alone.

Non-overlapping intervals are suggestive, not a test, and the comparisons that matter are paired - the same question answered by two systems. Those are reported in the accompanying paper using an exact McNemar test over discordant pairs. This report states the intervals and declines to convert them into significance claims they do not license.

8.3 Ablations

Configuration	Accuracy	Hallucination	Coverage	Disclosure

PRAXIS (full), nothing disabled	0.796	0.149	0.943	0.000
Unknown treated as false	0.745	0.188	0.952	0.000
Newest policy applied to every case	0.761	0.184	0.943	0.000
Asker's statements override records	0.690	0.094	0.748	0.000
Reasoning sees every record	0.737	0.208	0.943	0.867

Table 10. Each row below the first disables exactly one mechanism. Scored on the structured decision, so these columns cannot measure the verification gate, which acts on released wording.

Table 10 isolates each mechanism. Three-valued reasoning carries real weight: treating unknown as false costs 0.051 accuracy, because the system starts denying entitlements that were merely unrecorded. Authorization is the sharpest: removing it leaves accuracy broadly intact but raises the disclosure rate from 0.000 to 0.867 - the system answers well and leaks almost every protected record while doing so, which an accuracy-only evaluation would not reveal.

One row needs reading with care. Letting the asker's statements override records lowers hallucination to 0.094, below the full system - but coverage falls from 0.943 to 0.748. The configuration abstains its way to a better hallucination rate, which is exactly why coverage is reported beside every accuracy.

8.4 Transfer to unseen worlds

On the confirmatory split - five worlds the system had never seen, written after it was final and scored once - the LLM-free configuration transfers essentially unchanged: 0.812 against 0.796 on test, a difference well inside either interval. The parser configuration does not: accuracy falls from 0.945 to 0.835, and selective accuracy from 1.000 to 0.909.

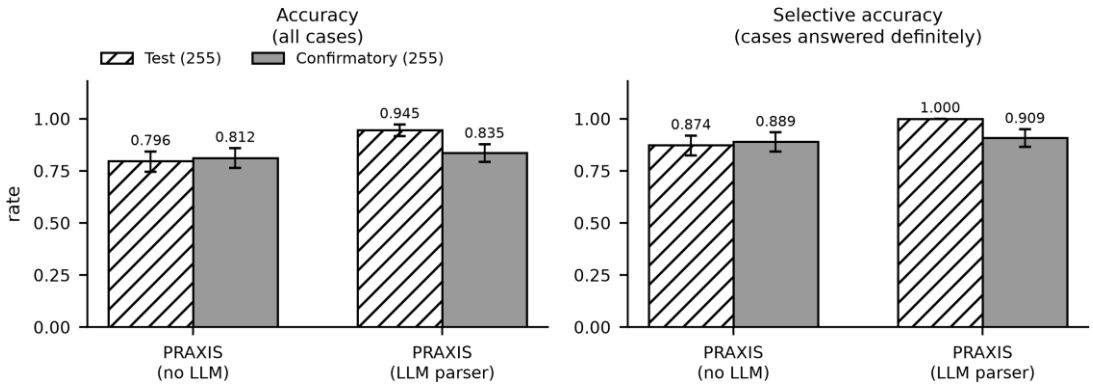


Figure 8. Test against confirmatory, both configurations, with 95% bootstrap intervals. The left panel is accuracy over all cases; the right is selective accuracy over the cases answered definitely. The LLM-free pair is flat; the parser pair is not.

Figure 8 shows the asymmetry, which is the more important of the two results in this section. The configuration with no language model on any path behaves the same on companies it has never seen, which is what one would expect of a rule engine: nothing in it was fitted to the test worlds. The configuration whose understanding step is a language model loses 0.110 accuracy when the companies change. The perfect selective accuracy on the test split is therefore not a reliability property of the architecture; it is a number that did not survive contact with new data, and reporting it without its confirmatory counterpart would be the single most misleading thing this report could do.

8.5 Secondary study: gating a model trained from scratch

A 4.3M-parameter decoder was trained from random initialization on a corpus of FIFA World Cup goal records, then made to generate freely while the symbolic layer decided what reached the reader. Of 1,056 checkable claims, the graph refutes **493** (0.467) and cannot confirm a further 40; gating releases the 523 supported ones, leaving coverage of 0.495.

Refuted and unverifiable claims are kept apart deliberately: the knowledge graph is incomplete, so a claim it has no record of is not thereby false. The gated rate of zero refuted claims holds **by construction** - the gate releases only what the graph confirms - and is not evidence of perfect verification. The informative quantities are the base refuted rate and the coverage that suppressing it costs: verification buys soundness with silence.

9. Limitations

- **Scope.** Results come from generated worlds with oracle gold answers, not real customer traffic, in a single domain whose governing rules can be written down. English only.
- **Question relevance is implemented only where understanding can decline.** The default encoder configuration abstains on none of the fifteen not-covered questions, because nearest-neighbour goal selection always returns a goal. The parser configuration now abstains on all fifteen. Carrying the fix to the default would mean giving the encoder negative catalogue entries - enumerating what the company does not do - which is open-ended. Section 7.2 demonstrates both halves.
- **Verifier correctness is unmeasured.** The verifier is correct by construction with respect to its own rule base; independent ground-truth labels would be needed to measure its precision and recall.
- **Authorship.** The rule base, world generators and oracle were written by one person. An independently authored evaluation is the natural next step.
- **Transfer.** Perfect selective accuracy on the test split does not generalize, as Section 8.3 shows.
- **Neither study speaks to open-domain factuality**, where no curated knowledge base exists for the gate to consult.
- **Statistical intervals treat questions as independent.** They are generated within worlds and share each world's policy parameters, so the reported intervals are narrower than a world-clustered analysis would give (Section 8.2).

9.1 External validity and deployment risk

The limitations above are properties of the evaluation. This section asks a different question: what would go wrong if this architecture were deployed against a real company's policies tomorrow? Three risks follow directly from how it works, and each is a consequence of the design rather than a defect to be patched.

- **The rule base becomes the ceiling, and the failure is silent** - every guarantee here is relative to rules someone wrote. A policy transcribed incorrectly produces answers that are confidently wrong and fully verified, because the verifier checks entailment from the rule base and not the rule base against the world. In this evaluation the rules were written by the same person who wrote the generator, so transcription error is systematically understated. A deployment would need the rule base reviewed by whoever owns the policy, and a route for them to contest an answer.
- **Coverage collapses where rules cannot be written** - the architecture answers what its rules decide and abstains otherwise. Where a real policy is vague, contested or exercised by human judgement, that is not a small loss of coverage but the normal case, and the system would abstain on a large fraction of genuine traffic. The honest deployment shape is therefore a triage layer in front of people, not a replacement for them, and the coverage figures here come from a domain deliberately chosen because its rules *can* be written down.
- **Relevance failures survive every check** - Section 7.2 shows the system answering a warranty question out of the replacement policy, with every applicable check passing. In an evaluation this is one scored failure. In deployment it is the most dangerous output the system can produce, because it carries real citations and a real record, and is therefore more persuasive than an unsupported guess would be. The mitigation implemented here works only on the parser path; on the default path there is none.

A fourth risk is procedural rather than technical. The evaluation reports coverage beside every accuracy precisely so that abstention cannot be mistaken for reliability, but a deployment dashboard that showed only hallucination rate would make the trust-statements ablation in Table 10 look like an improvement. Whatever is measured in production has to carry both numbers, for the same reason this report does.

10. Conclusion and Future Work

Removing the language model from the decision path, rather than asking it to audit itself, reduced hallucination from 0.302 to 0.149 while raising coverage from 0.462 to 0.943 and cutting latency by roughly two orders of magnitude. The reduction is not an artefact of refusing difficult questions. The latency comparison is not hardware-controlled: PRAXIS runs entirely on CPU, while the baselines wait on a served language model, so the gap measures the cost of needing a generator at all rather than two implementations on equal footing.

The problem addressed. Consequential questions are being answered by systems whose fluency is uncorrelated with their correctness. Retrieval changes what the answerer has read; it does not change who draws the inference, and a model handed the right clause can still assert a conclusion that clause does not license.

The architectural contribution, and what the experiments actually establish. The contribution is a division of authority: the language model interprets and may phrase, a rule engine decides, and a deterministic gate releases. The experiments establish three things. Symbolic reasoning matters on its own terms - treating unknown as false costs 0.051 accuracy, because the system begins denying entitlements that were merely unrecorded, and a two-valued engine cannot tell 'no' from 'nobody wrote it down'. Authorization matters more sharply than accuracy reveals - removing it leaves accuracy broadly intact while the disclosure rate goes from 0.000 to 0.867, so a system evaluated on accuracy alone would be scored as essentially unchanged while leaking almost every protected record. And coverage must accompany every hallucination figure, because one ablation in Table 10 lowers hallucination below the full system purely by abstaining more.

What is demonstrated, and what is proposed. Demonstrated, on the frozen test split and confirmed on unseen companies: the LLM-free configuration answers more questions than retrieval and more of them correctly, and it transfers. Demonstrated and unflattering: the parser configuration's perfect selective accuracy does not transfer, and the default configuration does not recognise questions its rule base does not cover. That last failure is structural rather than a tuning problem - verification establishes that a claim follows from evidence, not that it answers the question asked, and support and relevance are separate properties throughout this report. The architecture secures the first by construction and the second only where understanding is able to decline.

Proposed, not demonstrated. Carrying the relevance gate to the default configuration requires giving nearest-neighbour goal selection a null option it does not have, and the obvious route - negative catalogue entries - means enumerating what a company does not do, which is open-ended. An independently authored evaluation, in which the rule base, the world generator and the oracle are not written by one person, would test the verifier's correctness rather than assuming it. Raising coverage in the secondary study without reintroducing unsupported claims remains open. None of these is claimed as a result here.

11. References

[1] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, art. 248, pp. 1-38, 2023, doi: 10.1145/3571730.

[2] L. Huang et al., "A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions," ACM Trans. Inf. Syst., vol. 43, no. 2, 2025, doi: 10.1145/3703155.

[3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020.

[4] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: learning to retrieve, generate, and critique through self-reflection," in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[5] S. Dhuliawala et al., "Chain-of-verification reduces hallucination in large language models," in Findings Assoc. Comput. Linguistics: ACL 2024, pp. 3563-3578, doi: 10.18653/v1/2024.findings-acl.212.

[6] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: zero-resource black-box hallucination detection for generative large language models," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2023, pp. 9004-9017, doi: 10.18653/v1/2023.emnlp-main.557.

[7] S. Min et al., "FActScore: fine-grained atomic evaluation of factual precision in long form text generation," in Proc. EMNLP, 2023, pp. 12076-12100, doi: 10.18653/v1/2023.emnlp-main.741.

[8] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: empowering large language models with symbolic solvers for faithful logical reasoning," in Findings Assoc. Comput. Linguistics: EMNLP 2023, pp. 3806-3824, doi: 10.18653/v1/2023.findings-emnlp.248.

[9] T. X. Olausson et al., "LINC: a neurosymbolic approach for logical reasoning by combining language models with first-order logic provers," in Proc. EMNLP, 2023, pp. 5153-5176, doi: 10.18653/v1/2023.emnlp-main.313.

[10] L. Gao et al., "PAL: program-aided language models," in Proc. 40th Int. Conf. Mach. Learn. (ICML), PMLR, vol. 202, 2023, pp. 10764-10799.

[11] Z. Cheng et al., "Binding language models in symbolic languages," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[12] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: the 3rd wave," Artif. Intell. Rev., vol. 56, no. 11, pp. 12387-12406, 2023, doi: 10.1007/s10462-023-10448-w.

[13] S. C. Kleene, Introduction to Metamathematics. Amsterdam, The Netherlands: North-Holland, 1952.

12. Appendix: Source Code and Reproduction

Repository: https://github.com/A-Pravin02/praxis-neurosymbolic

Commit reported here: 1ec2a57 — the state of the code that produced every number in this report. The commit that carries this document is its child, since the document is written after the code it describes; both are on `master`.

Tested on: Windows 10.0.26200 (AMD64), Python 3.11.9. Nothing below needs a database, a GPU, a network connection or an API key; the encoder runs locally through ONNX Runtime and the benchmark reads committed records. That claim is exercised rather than asserted - the Table 5 results are re-derived by the first command, and the test suite invoked by the last is the database-free subset.

Setup. The only platform difference is the path to the virtual environment's executables:

```
git clone https://github.com/A-Pravin02/praxis-neurosymbolic && cd praxis-neurosymbolic
python -m venv backend/.venv

# Windows (PowerShell or Git Bash)
backend\.venv\Scripts\pip install -r backend/requirements.txt

# Linux or macOS
backend/.venv/bin/pip install -r backend/requirements.txt
```

Running the demo interface. Use the interpreter from that environment (`backend\.venv\Scripts\python` on Windows, `backend/.venv/bin/python` otherwise); the listings below write `python` for whichever applies:

```
python demo/server.py --warm
```

```
# then open http://127.0.0.1:8099
```

Reproducing the reported numbers. Each command checks a different thing, and each prints a pass or fail rather than requiring the output to be read against this document:

```
python scripts/independent_metrics.py  # re-derives every metric from raw records
python scripts/verify_claims.py        # checks the paper's claims against artifacts
python scripts/count_checks.py         # censuses the verification checks
python scripts/check_report.py         # checks this report's numbers and numbering

cd backend && python -m pytest -q -m no_db   # the database-free test suite
```

Rebuilding this document and its figures:

```
python scripts/make_architecture_figure.py --audit
python scripts/make_results_figure.py
python scripts/make_transfer_figure.py
python scripts/record_demo_suite.py    # re-records the ten sample cases
python scripts/make_project_report.py
```

The per-question records behind every reported number are committed under `eval_reports/research/`, so a clone can re-verify the results without re-running the experiments or loading a language model. Re-running the language-model baselines themselves does need a served model and is not required to check any figure in this report. Model checkpoints are excluded for size and are regenerated by the training scripts.